

ภาคผนวก ก

ตัวอย่างซอสโค้ดการทำงานของโปรแกรม

1. ซอร์สโค้ดในส่วนของการสร้างการเชื่อมต่อระหว่างโปรแกรมบนโทรศัพท์เคลื่อนที่และเซิร์ฟเวอร์

```
void CSocketEngine::ConnectL(TDesC8& aUsername, TDesC8& aPassword)
{
    if (iState == ENotConnected)
    {
        // Open a TCP socket
        iSockAddr.SetPort(KPortNumber);
        iSockAddr.Input(KServerIP);
        User::LeaveIfError(iSocket.Open(iSocketServer, KAfInet, KSockStream, KProtocolInetTcp));

        // Initiate socket connection
        iSocket.Connect(iSockAddr, iStatus);
        iState = EConnecting;

        // Start a timeout
        iTimer->After(KTimeOut);

        SetActive();
    }
}
```

เป็นส่วนที่จะทำการเชื่อมต่อไปยังเครื่อง เซิร์ฟเวอร์ โดยต้องทำการประกาศตัวแปร iSocket ชนิดของข้อมูลเป็น RSocket เพื่อใช้สำหรับเชื่อมต่อไปยัง Server โดยใช้ฟังก์ชัน Open() สำหรับติดต่อไปยัง SocketServer ซึ่งต้องประกาศเอาไว้ก่อน และกำหนดเลขพอร์ตการติดต่อในที่นี้ด้วย KAfInet ซึ่งจะถูกเชื่อมต่อด้วยฟังก์ชัน SetPort และ Input เพื่อระบุปลายทางที่ต้องการติดต่อ และพารามิเตอร์ KSockStream และ KProtocolInetTcp ถูกใช้เพื่อระบุว่าส่งผ่านโปรโตคอล TCP ด้วยข้อมูลแบบ Stream ตามลำดับ จากนั้นใช้คำสั่ง Connect สำหรับติดต่อไปยังเครื่องที่ต้องการโดยส่งตัวแปรแอดเดรสเข้าไปทำการ SetActive เพื่อรอรับการเชื่อมต่อ

2. ซอร์สโค้ดในส่วนของการส่งข้อมูลจากโปรแกรมบนโทรศัพท์เคลื่อนที่ไปยังเซิร์ฟเวอร์

```
void CSocketsWriter::SendNextPacket()
{
    if (iTransferBuffer.Length() > 0)
    {
        // Move data from transfer buffer to actual write buffer
        iWriteBuffer = iTransferBuffer;
        iTransferBuffer.Zero();
        iSocket.Write(iWriteBuffer, iStatus); // Initiate actual write

        // Request timeout
        iTimer->After(iTimeOut);
        SetActive();
        iWriteStatus = ESending;
    }
    else
    {
        iWriteStatus = EWaiting;
        iEngine->WroteProcess(KErrNone);
    }
}
```

ฟังก์ชัน IssueWriteL() จะทำหน้าที่ในการรับข้อมูลที่จะส่งมาใส่ใน Buffer ที่เตรียมไว้แล้วก่อน จากนั้นจึงเรียก ฟังก์ชัน SendNextPacket() ให้ทำงาน ฟังก์ชันนี้เมื่อทำการตรวจสอบ Buffer แล้วว่ามีข้อมูลที่ส่งจริง ก็จะทำการส่งด้วยฟังก์ชัน Write() จากนั้นก็จะทำการตั้งเวลา และทำการสั่ง SetActive() เพื่อรอผลจากการส่งข้อมูล ถ้าข้อมูลส่งได้ไม่มีปัญหาจะมีการส่งค่ากลับมาที่ iStatus เป็น KErrNone

3. ซอร์สโค้ดในส่วนของการรับข้อมูลของโปรแกรมบนโทรศัพท์เคลื่อนที่

```
void CSocketsReader::IssueRead()
{
    // Initiate a new read from socket into iBuffer
    __ASSERT_ALWAYS(!IsActive(), User::Panic(KPanicSocketEngineRead, ESocketBadState));
    iSocket.RecvOneOrMore(iBuffer, 0, iStatus, iDummyLength);
}
```

```
SetActive();
}
```

ฟังก์ชัน IssueRead() จะทำหน้าที่ในการรับข้อมูลที่ส่งมาจากเซิร์ฟเวอร์โดยใช้ฟังก์ชัน RecvOneOrMore() จากนั้นก็จะทำการ SetActive() เพื่อให้ทำการรอรับข้อมูลจากเซิร์ฟเวอร์

4. ซอร์สโค้ดในส่วนของการสร้างการเชื่อมต่อระหว่างโปรแกรมบนเซิร์ฟเวอร์กับโทรศัพท์เคลื่อนที่

```
void CMVBServerDlg::AcceptClient()
{
    int i;
    for (i=0; i<KUserMax; i++) {
        if (m_UserSocket[i].iSocketState == ENotConnected) {
            m_ListenSock.Accept(m_UserSocket[i]);
            m_UserSocket[i].SetSockAcceptedInfo();
            break;
        }
    }
    if (i >= KUserMax) {
        CAsyncSocket RejectSock; // New socket
        CString strInfo;
        strInfo = "Limit Over:";

        m_ListenSock.Accept(RejectSock);
        RejectSock.Send(LPCTSTR(strInfo), strInfo.GetLength());
        RejectSock.Close();
    }
}
```

เมื่อมีการร้องขอการเชื่อมต่อโปรแกรมจะทำการตรวจสอบก่อนว่ามี socket ในการเชื่อมต่อว่างเหลืออยู่หรือไม่ โดยสามารถกำหนดจำนวน socket ที่ทำการเชื่อมต่อได้จากตัวแปร KUserMax ถ้าว่างจะรับการเชื่อมต่อ นั้น โดยเรียกฟังก์ชัน SetSockAcceptedInfo() การเปลี่ยน state จาก ENotConnected เป็น Econnected

5. ซอร์สโค้ดในส่วนของการติดต่อและส่งข้อมูลระหว่างโปรแกรมบนเซิร์ฟเวอร์กับโทรศัพท์ที่เคลื่อนที่

```

void CASock::OnReceive(int nErrorCode)
{
    //-----declaration-----
    int buffer_size=512;
    char buffer[4096];
    nRecv = Receive(buffer,buffer_size);
    CString strShowClosePeer;

    //-----End declaration-----

    //Receive file data
    if(this->iSocketState==EReceivedHeader)//EReceivedHeader
    {
        //Write to file
        ReceiveFile(buffer,user,char_CategoryID,count_file,char_file_name,file_size,desc,topic,first,file_Re
cv,nRecv);
    }

    //Receive file name,file size and text
    if(this->iSocketState==ELogged)
    {
        ReceiveHeader(buffer,char_CategoryID,count_file,char_file_name,file_size,desc,topic);
    }

    //Receive all data from mobile complete
    if(this->iSocketState==EReceivedData)//EReceivedData
    {
        strShowClosePeer.Format("Complete.");
        ((CMVBServerDlg*)m_pDwnd)->m_displayList.AddString(strShowClosePeer);
        this->iSocketState=ELogged; //Reset state and value
        first=1;
    }

    //Verify user and password
    if(this->iSocketState==EConnected)
    {

```

```

        VerifyUserPasswd(buffer,user);
    }
}

```

เมื่อทำการเชื่อมต่อแล้วต่อไปจะเป็นการตรวจสอบรหัสผ่านและการส่งไฟล์ข้อมูล โดยตอนแรก state เป็น EConnected และเรียกฟังก์ชัน VerifyUserPasswd เพื่อตรวจสอบรหัสผ่าน ถ้ารหัสผ่านถูกต้อง state จะเปลี่ยนเป็น ELogged เมื่อทำการล็อกอินแล้วจะเข้าสู่การส่งเฮดเดอร์ โดยเรียกฟังก์ชัน ReceiveHeader เพื่อรับชื่อไฟล์และขนาดของไฟล์ จากนั้น state จะเปลี่ยนเป็น EReceivedHeader แล้วจะเรียกฟังก์ชัน ReceiveFile เพื่อรับข้อมูลไฟล์นำมาเขียนเป็นไฟล์ เมื่อรับข้อมูลแต่ละไฟล์เสร็จจะทำการอัปเดตฐานข้อมูล เมื่อรับข้อมูลครบทั้งหมดจะเปลี่ยน state เป็น EReceivedData แล้วกลับมาสู่สถานะ ELogged อีกครั้ง